

Sprawozdanie: Funkcje skrótu

Jakub Biernat 160248

1. Kod aplikacji:

```
import hashlib
from string import ascii_letters, digits
import time
import matplotlib.pyplot as plt
from secrets import choice

ALPHABET = ascii_letters + digits
def generate_hash(text, function):
    hash = hashlib.new(function)
    hash.update(text.encode())
    return hash.hexdigest()

def compare_hashes(texts):
    functions = ['md5', 'sha1', 'sha224', 'sha256', 'sha384', 'sha512', 'sha3_224', 'sha3_256', 'sha3_384', 'sha3_512']

    input_bits = []
    times = {name: [] for name in functions}
    hash_lengths = {name: [] for name in functions}

    for text in texts:
        input_bits.append(len(text.encode()) * 8)
        for function in functions:
            start = time.perf_counter()
            hash = generate_hash(text, function)
            stop = time.perf_counter()

            times[function].append(stop - start)
            hash_lengths[function].append(len(hash) * 4)

    plt.figure(figsize=(12, 6))

    ax1 = plt.subplot(1, 2, 1)
    for function in functions:
        ax1.plot(input_bits, hash_lengths[function], label=function)

    ax1.set_xlabel("Długość danych wejściowych (bity)")
    ax1.set_ylabel("Długość wygenerowanego hashu (bity)")
    ax1.set_title("Długość hashu w zależności od długości danych")
    ax1.legend()
    ax1.grid()
    ax1.ticklabel_format(style='plain', axis='x')

    ax2 = plt.subplot(1, 2, 2)
    for function in functions:
        ax2.plot(input_bits, times[function], label=function)

    ax2.set_xlabel("Długość danych wejściowych (bity)")
    ax2.set_ylabel("Czas generowania hashu (sekundy)")
    ax2.set_title("Czas generowania hashu w zależności od długości danych")
    ax2.legend()
    ax2.grid()
    ax2.ticklabel_format(style='plain', axis='x')

    plt.tight_layout()
    plt.savefig("Wykresy.png", dpi=300)
    plt.show()
```

```

def find_collision(function, prefix_bits):
    seen = {}
    attempts = 0

    while True:
        attempts += 1
        text = ''.join(choice(ALPHABET) for _ in range(16))
        hash = generate_hash(text, function)

        bit_hash = bin(int(hash, 16))[2:].zfill(len(hash) * 4)
        prefix = bit_hash[:prefix_bits]

        if prefix in seen:
            return attempts, seen[prefix], text, prefix
        else:
            seen[prefix] = text

def bit_difference(hash1, hash2):
    bit_hash1 = bin(int(hash1, 16))[2:].zfill(len(hash1) * 4)
    bit_hash2 = bin(int(hash2, 16))[2:].zfill(len(hash2) * 4)
    return sum(bit1 != bit2 for bit1, bit2 in zip(bit_hash1, bit_hash2))

def flip_bit(text):
    bytes = bytearray(text, 'utf-8')

    byte_index = choice(range(len(bytes)))
    bit_index = choice(range(8))
    bytes[byte_index] ^= (1 << bit_index)

    return bytes.decode('utf-8', errors='ignore')

def avalanche_test(function, tests_num):
    results = []

    for _ in range(tests_num):
        text = ''.join(choice(ALPHABET) for _ in range(16))
        hash1 = generate_hash(text, function)

        modified_text = flip_bit(text)
        hash2 = generate_hash(modified_text, function)

        diff = bit_difference(hash1, hash2)
        results.append(diff / (len(hash1) * 4))

    avg = sum(results) / len(results)

    return avg

```

```

def main():
    #Podpunkt 2
    texts = ["a" * i for i in range(100000, 1000000, 100000)]
    compare_hashes(texts)

    #Podpunkt 3
    print(f"Wygenerowany hash dla słowa 'Owad': {generate_hash('Owad', 'md5')}")
    #Po sprawdzeniu w wyszukiwarce, hash ten jest powszechnie znany

    #Podpunkt 4
    #MD5 nie jest funkcją bezpieczną, ponieważ znaleziono dla niej kolizje, pierwsze już w 2004 roku.
    #Dodatkowo MD5 jest bardzo szybka, co ułatwia ataki brute-force.

    # Podpunkt 5
    attempts, text1, text2, prefix = find_collision("sha3_512", 12)
    print(f"\nKolizja na pierwszych 12 bitach (SHA-3-512) znaleziona po {attempts} próbach:")
    print(f"Tekst 1: {text1}")
    print(f"Tekst 2: {text2}")
    print(f"Wspólne 12 bitów: {prefix}")
    print(f"Hash 1: {generate_hash(text1, 'sha3_512')}")
    print(f"Hash 2: {generate_hash(text2, 'sha3_512')}")

    # Podpunkt 6
    print("\nAvalanche test (SHA-3-512):")
    tests_num = 200
    print(f"Po {tests_num} próbach, średni procent zmienionych bitów po zmianie jednego bitu na wejściu: "
          f"{avalanche_test('sha3_512', tests_num)}")
    #Test się powiódł

main()

```

2. Omówienie sposobu implementacji

Wybrany język programowania: Python

Użyte biblioteki:

- hashlib- obliczanie skrótów kryptograficznych
- time- obliczanie czasu działania funkcji
- matplotlib- wizualizacja wyników
- string- alfabet do generowania losowych danych wejściowych
- secrets- funkcja losująca do generowania losowych danych wejściowych

Główne elementy implementacji:

- generate_hash() – generuje hash dla podanego tekstu i funkcji
- compare_hashes() – porównuje: długości hashy oraz czas ich obliczania dla różnych funkcji
- find_collision() – wyszukuje kolizje dla wskazanej ilości pierwszych bitów
- avalanche_test() – sprawdza efekt lawinowy funkcji
- bit_difference()- zliczanie różnicy bitów
- flip_bit()- zamiana losowego bitu

3. Określenie roli soli w tworzeniu skrótów

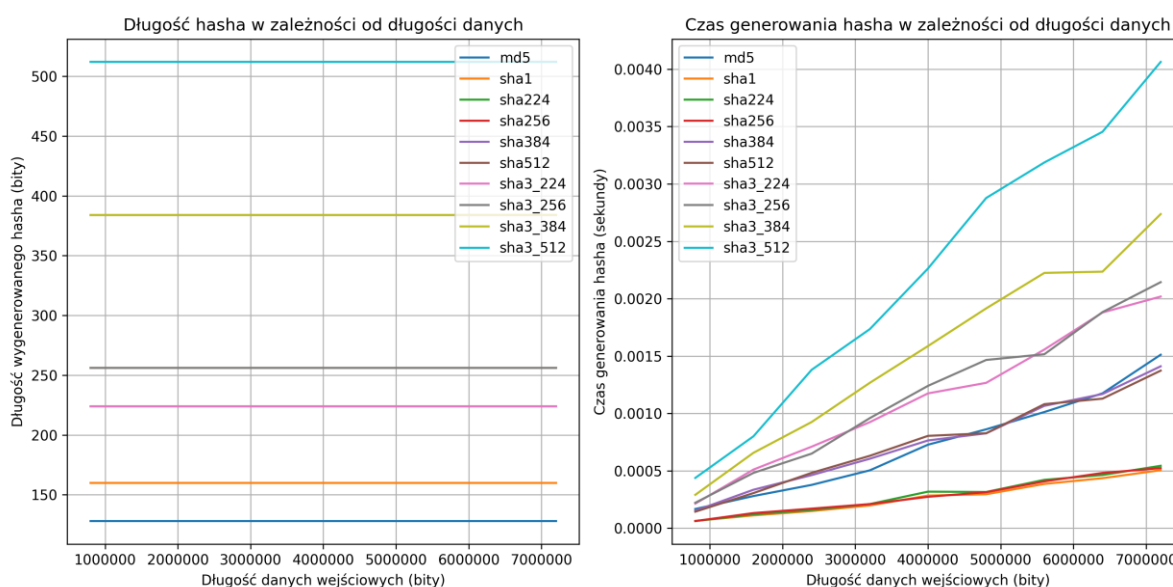
Sól to losowy ciąg dodawany do danych przed hashowaniem, dzięki czemu dwa identyczne hasła mają różne skróty, co zwiększa bezpieczeństwo i odporność na ataki.

4. Odpowiedź oraz jej uzasadnienie na pytanie postawione w pkt. 4

MD5 nie jest funkcją bezpieczną, ponieważ znaleziono dla niej kolizje, pierwsze już w 2004 roku. Dodatkowo MD5 jest bardzo szybka, co ułatwia ataki brute-force.

5. Zestawienie uzyskanych wyników wraz ze stosownymi wnioskami

- Eksperyment z porównaniem długości ciągów wejściowych i czasu działania różnych funkcji skrótu dla zróżnicowanej długości danych wejściowych:



Wnioski: Niezależnie od długości danych wejściowych, funkcje skrótu generują ciąg o takiej samej długości, natomiast w ogólności im funkcja jest bezpieczniejsza, tym jest wolniejsza.

- Eksperyment z badaniem czy hash funkcji MD5 jest znany:
Wygenerowany hash dla słowa 'Owad': 37b45783ea13c438319c662d7f6fb143
Po sprawdzeniu w wyszukiwarce, hash ten jest powszechnie znany. Skróty z krótkich haseł nie są bezpieczne.

- Eksperyment z badaniem kolizji:

Kolizja na pierwszych 12 bitach (SHA-3-512) znaleziona po 87 próbach:

Tekst 1: kSgojXWYu8M0hzdI

Tekst 2: EG1BbyZfjCKfGM4b

Wspólne 12 bitów: 101110100010

Hash 1:

*ba2c8a30757fbdb053ac6b58b8379e950c0fa7d7e21d7b3bdf5387a4613aec5e0
76ea991da668efc5bf24b57fc82448e3748b32ed602f21dbbaafb355a770d8d*

Hash 2:

*ba205e0fccd21f4f1aacb62e306a320868c06884f7c7b480f43f76ce6df8574c648
4f597f0b7d06c24c1d4f93a095ff508f88de1fd81f06795286777f1743b41*

- Eksperyment z badaniem efektu lawinowego:

Avalanche test (SHA-3-512):

Po 200 próbach, średni procent zmienionych bitów po zmianie jednego bitu na wejściu: 0.498779296875

Eksperyment się powiódł